P0009r5 : Polymorphic Multidimensional Array Reference

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P0009r5
Date:	2018-02-10
Reply-to:	hcedwar@sandia.gov
Author:	H. Carter Edwards
Contact:	hcedwar@sandia.gov
Author:	Daniel Sunderland
Contact:	dsunder@sandia.gov
Author:	David Hollman
Contact:	dshollm@sandia.gov
Author:	Christian Trott
Contact:	crtrott@sandia.gov
Author:	Mauro Bianco
Contact:	mbianco@cscs.ch
Author:	Ben Sander
Contact:	ben.sander@amd.com
Author:	Athanasios Iliopoulos
Contact:	athanasios.iliopoulos@nrl.navy.mil
Author:	John Michopoulos
Contact:	john.michopoulos@nrl.navy.mil
Author:	Daniel Sunderland
Contact:	dsunder@sandia.gov
Audience:	Library Working Group (LWG)
URL:	https://github.com/kokkos/array_ref/blob/master/proposals/P0009.rst

1 Revision History

1.1 P0009r0

Original multidimensional array reference paper with motivation, specification, and examples.

LEWG feedback `view` is not an acceptable name, bikeshed names: view , span , array_ref , slice , array_view , ref , array_span , basic_span , object_span , field

1.2 P0009r1

Revised with renaming from `view` to `array_ref` and allow unbounded rank through variadic arguments.

1.3 P0009r2

Adding details for extensibility of layout mapping.

Move motivation, examples, and relaxed incomplete array type proposal to separate papers.

- P0331 : Motivation and Examples for Polymorphic Multidimensional Array
- P0332 : Relaxed Incomplete Multidimensional Array Type Declaration

1.4 P0009r3

Oulu-2016 LEWG strawpoll: Move iterator from this paper to a subsequent paper.

Oulu-2016 LEWG feedback: http://wiki.edg.com/bin/view/Wg21oulu/P0009

- Array extents specification mechanism options are either-or, not both.

- List potential names for LEWG and/or LWG todo bikeshedding.
- Clearly & concisely note difference between multidimensional array versus language's array-of-array-of-array...
- Actual specification of reference type (and others), not "typically is" vagueness.
- Future directions / extensibility section regarding `Properties...`

The domain space specification *preferred* and *undesirable* mechanisms changed from accepting both to accepting only one.

Tighten up domain, codomain, and domain -> codomain mapping specifications.

Consistently use *extent* and *extents* for the multidimensional index space.

More LEWG name bikeshedding: sci_span , numeric_span , multidimensional_span , multidim_span , md_span , vla_span , multispan , multi_span

1.5 P0009r4 : For 2017-11-Albuquerque Meeting

Changes from P0009r3:

- Rename to `mdspan`, multidimensional span, to align with P0122r5 `span`.
- Move preferred array extents mechanism to appendix
- Align with P0122r5 span
- Expose codomain as a std::span
- Elaborate layout mapping

Requested full-LEWG straw polls at Albuquerque Nov'2017 meeting:

- Acceptability of exclusively using signed integer `index_type` and omitting the traditional container unsigned integer `size_type`.
- Embedding the domain-to-codomain mapping observers within the `mdspan` class or moving these into a `mdspan::mapping` class.
- Given authors' update in response to previous two straw polls, is this ready to advance to LWG?
- If not, what specific modifications are required for consensus?

1.6 P0009r5 : For 2018-03-Jacksonville Meeting

LEWG review at 2017-11-Albuquerque meeting Unanimous to forward to LWG with editorial and straw poll changes. All changes have been applied except allowing `span<int-type[N]>` indexing value for which there was weak support and no proven need. This could be added in a subsequent paper.

Changes from P0009r4

- Removed nullptr constructor
- Added constexpr to indexing operator
- Indexing operator requires that rank() == sizeof...(indices)
- Fixed typos in examples and moved them to appendix
- Converted note on how extentions to access properties may cause reference to be a proxy type to an see_below to make it normative

2 Description

The proposed polymorphic multidimensional array reference (`mdspan`) defines types and functions for mapping indices from a **multidimensional index space** (the domain) to members of a contiguous span of objects (the codomain). A multidimensional index space is defined as the Cartesian product of integer extents, **[0..N0) X [0..N1) X [0..N2) ...**

This **layout mapping** is one *property* of the `mdspan` that may be specified through a template parameter. The intent is that *properties** are an extensible set of options for multi-index mapping and member access. For example, bounds checking the input multi-index versus the multdimensional extents or accessing members through an atomic interface. The recent Accessors paper (P0367) introduces a rich set of potential access properties.

A multidimensional array is not an array-of-array-of-array-of-array...

The multidimensional array abstraction has been fundamental to numerical computations for over five decades. However, the C/C++ language provides only a one dimensional array abstraction which can be composed into array-of-array-of-array... types. While such types have some similarity to multidimensional arrays they do not provide adequate multidimensional array

functionality of this proposal. Two critical functionality differences are (1) multiple dynamic extents and (2) polymorphic mapping of multi-indices to member objects.

3 Multidimensional Array and Subarray Proposal

3.1 Add to same section and header as span

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

  inline constexpr ptrdiff_t dynamic_extent = -1 ; // Revise to add inline

  template< typename DataType , typename ... Properties >
  class mdspan ;

  template< ptrdiff_t ... StaticExtents >
  class extents ;

  class layout_right ;
  class layout_left ;
  class layout_stride ;

  template< ptrdiff_t ... LHS , ptrdiff_t ... RHS >
  constexpr bool operator == ( extents<LHS...> const & lhs , extents<RHS...> const & rhs ) ;

  template< ptrdiff_t ... LHS , ptrdiff_t ... RHS >
  constexpr bool operator != ( extents<LHS...> const & lhs , extents<RHS...> const & rhs ) ;

  // return type of subspan free function is an mdspan
  template< typename DataType , typename ... Properties , typename ... SliceSpecifiers >
    // for exposition only:
    detail::subspan_deduction_t< mdspan<DataType,Properties...>,SliceSpecifiers...>
  subspan( mdspan< DataType, Properties ... > const & , SliceSpecifiers ... ) noexcept;

  // tag supporting subspan
  struct all_type {};
  inline constexpr all_type all = all_type{};
}}}
```

The `mdspan` class maps a multi-index within a multi-index **domain** to a reference to the **codomain**, defined by a span of objects.

The `subspan` free function generates an `mdspan` with a domain contained within the input `mdspan` domain and codomain contained within the input `mdspan` codomain.

The *detail::subspan_deduction_t* template class is not proposed and appears for exposition only. An implementation metafunction of this form is necessary to deduce the specific `mdspan` return type of the `subspan` function.

3.2 template class `mdspan`

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

template <typename DataType, typename... Properties>
class mdspan {
public:
  // domain and codomain types

  using element_type    = typename remove_all_extents_t<DataType> ;
  using value_type      = typename remove_cv_t< element_type > ;
  using index_type      = ptrdiff_t ;
  using difference_type = ptrdiff_t ;
  using pointer         = element_type * ;
  using reference       = element_type & ;

  // Standard constructors, assignments, and destructor

  ~mdspan() noexcept ;

  constexpr mdspan() noexcept ;
  constexpr mdspan(mdspan&&) noexcept = default ;
  constexpr mdspan(mdspan const&) noexcept = default ;
```

```
  mdspan& operator=(mdspan&&) noexcept = default ;
  mdspan& operator=(mdspan const&) noexcept = default ;

  // Constructor and assignment for assignable mdspan

  template <typename UType, typename ... UProp>
  constexpr mdspan( mdspan<UType, UProp...> const& ) noexcept;

  template <typename UType, typename ... UProp>
  mdspan& operator=( mdspan<UType, UProp...> const& ) noexcept;

  // Wrapping constructors

  template< class ... IndexType >
  explicit constexpr mdspan(pointer, IndexType ... DynamicExtents ) noexcept;

  template< class ... IndexType >
  explicit constexpr mdspan(std::span<element_type>, IndexType ... DynamicExtents ) noexcept;

  template< class IndexType , size_t N >
  explicit constexpr mdspan(pointer, std::array<IndexType,N> const & DynamicExtents ) noexcept ;

  template< class IndexType , size_t N >
  explicit constexpr mdspan(std::span<element_type>, std::array<IndexType,N> const & DynamicExtents ) noexcept ;

  // mapping domain multi-index to access codomain member

  constexpr reference operator[]( index_type ) const noexcept; // requires rank() == 1

  template< class ... IndexType >
  constexpr reference operator()( IndexType ... indices ) const noexcept;

  template< class IndexType , size_t N >
  constexpr reference operator()( std::array<IndexType,N> const & indices ) const noexcept;

  // observers of the index space domain

  static constexpr int rank() noexecept ;
  static constexpr int rank_dynamic() noexecept ;

  static constexpr index_type static_extent(int) noexecept ;

  constexpr index_type extent(int) const  noexecept ;

  constexpr index_type size() const  noexecept ;

  // observers of the codomain:

  constexpr std::span<element_type> span() const  noexecept ;

  template< class ... IndexType >
  static constexpr index_type required_span_size( IndexType ... DynamicExtents );

  template< class ... IndexType , size_t N >
  static constexpr index_type required_span_size( std::array<IndexType,N> const & DynamicExtents );

  // observers of the mapping : domain -> codomain

  using layout = /* extracted from Properties... */ ;

  static constexpr bool is_always_unique     = /* layout */ ;
  static constexpr bool is_always_contiguous = /* layout */ ;
  static constexpr bool is_always_strided    = /* layout */ ;

  constexpr bool is_unique() const ;
  constexpr bool is_contiguous() const ;
  constexpr bool is_strided() const ;

  constexpr index_type stride(int) const ;

private:
  // exposition only
  typename layout::mapping< StaticExtents... > mapping ;
  pointer_type                                 ptr ;
};

}}}
```

3.2.1 Template arguments

```
template <typename DataType, typename... Properties> class mdspan
```

`DataType`

Requires: Is a non-array type denoting the element type of the array.

`Properties...`

Effects: The domain index space rank, static extents, and identification of dynamic extents is determined from the `extents` member of the property pack. The domain to codomain layout mapping is determined from the layout member of the property pack.

3.2.2 Fundamental Types

```
using element_type = typename remove_all_extents_t<DataType> ;
using value_type   = typename remove_cv_t<element_type> ;
using reference    = element_type & ;
using pointer      = element_type * ;
```

[If `std::is_const<element_type>` then references to codomain members are const. Extensions to access properties may cause `reference` to become a proxy type (see Appendix: Reference is potentially a proxy)]

```
using index_type      = ptrdiff_t ;
using difference_type = ptrdiff_t ;
```

[Note: Integral types for dimensions and indexing are signed integers to avoid casting unsigned-to-signed for loop bounds and improve opportunities for optimizing loops. --end note]

3.2.3 Domain Observers

The multi-index domain space is the Cartesian product of the extents: `[0..extent(0)) X [0..extent(1)) X ... X [0..extent(rank()-1))`. Each extent may be statically (at compile time) or dynamically (at runtime) specified.

```
static constexpr int rank();
```

Returns: Rank of the multi-index domain.

```
static constexpr int rank_dynamic();
```

Returns: number of extents that are dynamic.

```
static constexpr index_type static_extent(int r);
```

Requires: `0 <= r`

Returns: If `0 <= r < rank()` the statically specified extent. A statically declared extent of `dynamic_extent` denotes that the extent is dynamic. If `rank() <= r` then `static_extent(r) == 1`.

```
constexpr index_type extent(int r) const ;
```

Requires: `0 <= r`

Returns: If `0 <= r < rank()` the extent of coordinate `r`. If `rank() <= r` then `extent(r) == 1`.

```
constexpr index_type size() const ;
```

Returns: product of `extent(r)` where `0 <= r < rank()`.

3.2.4 Codomain Observers

Not all members of the codomain may be accessible through the layout mapping; i.e., the range of the mapping is contained within the codomain but may not be equal to the codomain.

```
constexpr std::span<element_type> span() const ;
```

Returns: An `std::span` for the codomain.

```
template< class ... IndexType >
static constexpr index_type required_span_size( IndexType ... DynamicExtents );
```

Requires:

- `rank_dynamic() == sizeof...(DynamicExtents)`
- `is_integral_type_v<IndexType>...`
- Denote the ith coordinate of `DynamicExtents...` as denoted as `DynamicExtents[ith]` then:
- `0 <= DynamicExtents[ith]` for `0 <= ith < rank_dynanic()`

Returns: The minimum size of the codomain to support the multi-index domain defined by the merging of `DynamicExents` with `StaticExtents`.

```
template< class ... IndexType , size_t N >
static constexpr index_type required_span_size( std::array<IndexType,N> const & DynamicExtents );
```

Requires:

- `rank_dynamic() == N`
- `is_integral_type_v<IndexType>...`
- `0 <= DynamicExtents[ith]` for `0 <= ith < rank_dynanic()`

Returns: The minimum size of the codomain to support the multi-index domain defined by the merging of `DynamicExents` with `StaticExtents`.

3.2.5 Constructors, assignments, destructor

```
constexpr mdspan();
```

Effect: Construct a *null* `mdspan` with codomain `span() == std::span<element_type>()` and `extent(r) == 0` for all dynamic extents.

```
template< typename UType , typename ... UProperties >
constexpr mdspan( mdspan< UType , UProperties ... > const & ) noexcept
template< typename UType , typename ... UProperties >
mdspan & operator = ( mdspan< UType , UProperties ... > const & ) noexcept
```

Requires: Given `using V = mdspan<DataType,Properties...>` and `using U = mdspan<UType,UProperties...>` then

`is_assignable<V::pointer,U::pointer>`,
`V::rank() == U::rank()`,
`V::static_extent(r) == U::static_extent(r)` or `V::static_extent(r) == std::dynamic_extent` for `0 <= r < V::rank()`,
compatibility of layout mapping

Effect: `* this` has equal domain, equal codomain, and equivalent mapping.

```
template< class ... IndexType >
constexpr mdspan( pointer ptr , IndexType ... DynamicExtents) noexcept
```

Requires:

- `sizeof...(DynamicExtents) == rank_dynamic()`
- `is_integral_type_v<IndexType>...`
- The ith coordinate of `DynamicExtents...`, denoted as `DynamicExtents[ith]`, is `0 <= DynamicExtents[ith]`.
- The span of elements denoted by `[ ptr , ptr + required_span_size(DynamicExtents...) )`, shall be a valid contiguous span of elements.

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents equal to `DynamicExtents...` and codomain equal to `std::span<element_type>( ptr , required_span_size(DynamicExtents...) )`

```
template< class IndexType , size_t N >
constexpr mdspan( pointer ptr , std::array<IndexType,N> const & DynamicExtents) noexcept
```

Requires:

- `N == rank_dynamic()`
- `is_integral_type_v<IndexType>...`
- `0 <= DynamicExtents[ith]`
- The span of elements denoted by `[ ptr , ptr + required_span_size(DynamicExtents) )`, shall be a valid contiguous span of elements.

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents equal to `DynamicExtents[ith].` and codomain equal to `std::span<element_type>( ptr , required_span_size(DynamicExtents) )`

```
template< class ... IndexType >
```

```
constexpr mdspan( std::span<element_type> s , IndexType ... DynamicExtents) noexcept
```

Requires:

- `sizeof...(DynamicExtents) == rank_dynamic()`
- `is_integral_type_v<IndexType>...`
- The ith coordinate of `DynamicExtents...`, denoted as `DynamicExtents[ith]`, is `0 <= DynamicExtents[ith]`
- `required_span_size(DynamicExtents...) <= s.size()`

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents equal to `DynamicExtents...` and codomain equal to `std::span<element_type>( ptr , required_span_size(DynamicExtents...) )`

```
template< class IndexType , size_t N >
constexpr mdspan( std::span<element_type> s , std::array<IndexType,N> const & DynamicExtents) noexcept
```

Requires:

- `N == rank_dynamic()`
- `is_integral_type_v<IndexType>...`
- `0 <= DynamicExtents[ith]`
- `required_span_size(DynamicExtents) <= s.size()`

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents equal to `DynamicExtents[ith]` and codomain equal to `std::span<element_type>( ptr , required_span_size(DynamicExtents[ith]) )`

3.2.6 Mapping domain multi-index to access elements in the codomain

```
reference operator[]( index_type index ) const noexcept
```

Requires: `rank() == 1` and `0 <= i < extent(0)`

Returns: A `reference` to the element mapped to by `index`.

```
template< class ... IndexType >
reference operator()( IndexType ... indices ) const noexcept
```

Requires: `indices` is a multi-index in the domain:

- `rank() == sizeof...(IndexType)`
- The ith coordinate of `indices...`, denoted as `indices[ith]`, is in the domain: `0 <= indices[ith] < extent(ith)`.

Returns: A `reference` to the element mapped to by `indices...`.

Remark: Optimization of the mapping operator is a critical feature of the multidimensional array implementation. Recommended optimizations include:

- Rank-specific overloads to better enable optimization of the member access operator.
- Inlining of a `constexpr` multi-index mapping expression that is **not** included in an optimizer's inlining budget.
- Compile-time evaluation statically determined portions of multi-index mapping expression.

```
template< class IndexType , size_t N >
reference operator()( std::array<IndexType,N> const & indices ) const noexcept
```

Requires: `indices` is a multi-index in the domain:

- `rank() == N`
- `0 <= indices[ith] < extent(ith)`.

Returns: A `reference` to the element mapped to by `indices...`.

Remark: Optimization of the mapping operator is a critical feature of the multidimensional array implementation. Recommended optimizations include:

- Rank-specific overloads to better enable optimization of the member access operator.
- Inlining of a `constexpr` multi-index mapping expression that is **not** included in an optimizer's inlining budget.
- Compile-time evaluation statically determined portions of multi-index mapping expression.

3.2.7 Mapping Observers

```
using layout = /* implementation deduces from Properties... */ ;
```

Identification of the layout mapping. If `Properties...` does not include a layout property then `layout` is `layout_right` denoting the traditional C/C++ mapping.

```
static constexpr bool is_always_unique =
constexpr bool is_unique() const noexcept ;
```

A layout mapping is *unique* if each multi-index in the domain is mapped to a unique member in the codomain.

```
static constexpr bool is_always_contiguous =
constexpr bool is_contiguous() const noexcept ;
```

A layout mapping is *contiguous* if the codomain elements accessed through the layout mapping form a contiguous span.

A layout mapping that is both unique and contiguous is *bijective* and has `size() == span().size()`.

```
static constexpr bool is_always_strided =
constexpr bool is_strided() const noexcept ;
```

A *strided* layout has constant striding between multi-index coordinates. Let `A` be an `mdspan` and `indices_V...` and `indices_U...` be multi-indices in the domain space such that all coordinates are equal except for the *ith* coordinate where `indices_V[ith] = indices_U[ith] + 1`. Then `stride(ith) = distance(& A(indices_V...) - & A( indices_U... )` is constant for all coordinates.

```
template< typename IntegralType >
constexpr index_type stride( IntegralType index ) const noexcept
```

Requires: `is_strided()`.

Returns: When `r < rank()` the distance between members when the index of coordinate `r` is incremented by one, otherwise 0.

3.3 template class `extents`

One of the valid members of an `mdspan` `Properties...` pack is an instantiation of template class `extents`. This property declares the rank and static extents of the `mdspan` type. Example:

```
using tensor = std::mdspan<double,std::extents<std::dynamic_extent,std::dynamic_extent,std::dynamic_extent>> ;
```

Note: A preferred, concise, and intuitive syntax for declaring the multidimensional index space of an `mdspan` is proposed in P0332.

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

template< ptrdiff_t ... StaticExtents >
class extents {
public:

  using index_type = ptrdiff_t ;

  // observers of the index space domain:
  // [0..extent(0))X[0..extent(1))X...X[0..extent(rank()-1))

  static constexpr int rank() noexcept ;
  static constexpr int rank_dynamic() noexcept ;

  static constexpr index_type static_extent(int) noexcept ;

  constexpr index_type extent(int) const noexcept ;

  constexpr index_type size() const noexcept ;

  // constructors/assignment/destructor

  ~extents() = default ;
  constexpr extents();
  constexpr extents(extents const &) = default ;
  constexpr extents(extents &&) = default ;
  extents & operator = (extents const &) noexcept = default ;
  extents & operator = (extents &&) noexcept = default ;

  template< class ... IndexType >
```

```
  constexpr extents( IndexType ... DynamicExtents ) noexcept ;
};

}}}
```

3.4 subspan

```
template< typename DataType , typename ... Properties , typename ... SliceSpecifiers >
  // for exposition only:
  detail::subspan_deduction_t<mdspan<DataType,Properties...>,SliceSpecifiers...>
subspan( mdspan< DataType, Properties ... > const & U , SliceSpecifiers ... slices ) noexcept;
```

The `detail::subspan_deduction_t` is for exposition only to indicate that the implementation will require a metafunction to deduce the resulting `mdspan` type from `U` and `slices...`.

Let the *ith* member of `slices...` be denoted by `slices[ith]`.

Let an *integral range* be denoted by any of the following.

- an `initializer_list<T>` of integral type `T` and size 2
- a `pair<T,T>` of integral type `T`
- a `tuple<T,T>` of integral type `T`
- an `array<T,2>` of integral type `T`
- `all` to denote the range `[0 .. U.extent(ith))`

If `slices[ith]` is an integral range then let `begin(slices[ith])` be the beginning of the integral range `end(slices[ith])` be the end of the integral range. If `slices[ith]` is an integral value then let `begin(slices[ith]) == slices[ith]` and `end(slices[ith]) == slices[ith]+1`.

Requires:

- `U.rank() == sizeof...(slices)`.
- Each member of the `slices...` pack is either an *integral range* or an *integral value*.
- `0 <= begin(slices[ith]) <= end(slices[ith]) <= U.extent(ith)`.

Returns: An `mdspan V` with a domain contained within the domain of `U`, codomain contained within the codomain of `U`, `V.rank()` is the number of integral ranges in `slices...`, `U( begin(slices)... )` refers to the same codomain member refered to by the mapping the zero-index of `V`, each integral value in `slices...` contracts the corresponding extent of `U`.

3.4.1 Slice Specifier with Static Extent

The proposed `initializer_list`, `pair`, `tuple`, and `array` slice specifier types define dynamic extents. When the `all` slice specifier references a static extent then the subspan's corresponding extent should be static as well. When the extent of a slice specifier is statically known there should be a slice specifier type to explicitly express this knowledge. Such a static extent slice specifier type is to-be-done.

3.5 Layout properties

An `mdspan` maps multi-indices from the domain to reference elements in the codomain by composing a *layout mapping* with a span of elements. The layout mapping is an extension point such that an `mdspan` may be instantiated with non-standard layout mappings.

3.5.1 Predefined, Standard Layouts

The `layout_right` property denotes the C/C++ standard multidimensional array index mapping where the right-most extent is stride one and strides increase right-to-left as the product of extents.

The `layout_left` property denotes the FORTRAN standard multidimensional array index mapping where the left-most extent is stride one and strides increase left-to-right as the product of extents.

The `layout_stride` property denotes a multidimensional array index mapping with arbitrary strides for each extent. This is the layout for subarrays that are not contiguous.

The three standard layouts have the following layout mapping traits.

`layout_right` ; i.e., the C/C++ standard layout

`is_always_unique == true`
`is_always_contiguous == true`
`is_always_strided == true`
When `0 < rank()` then `stride(rank()-1) == 1`.
When `1 < rank()` then `stride(r-1) = stride(r) * extent(r)` for `0 < r < rank()`..

For rank-two arrays (a.k.a., matrices) this is also known as *row major* layout.

`layout_left` ; i.e., the FORTRAN standard layout

`is_always_unique == true`
`is_always_contiguous == true`
`is_always_strided == true`
When `0 < rank()` then `stride(0) == 1`.
When `1 < rank()` then `stride(r) = stride(r-1) * extent(r-1)` for `0 < r < rank()`..

For rank-two arrays (a.k.a., matrices) this is also known as *column major* layout.

`layout_stride` ; i.e., an arbitrary **strided** layout

`is_always_unique == false`
`is_always_contiguous == false`
`is_always_strided == true`

3.5.2 Concept for Extensible Layout Mapping

A *layout* class conforms to the following interface such that an `mdspan` can compose the layout mapping with its `mdspan` codomain member reference generation.

```
class layout_property /* exposition only */ {
public:

  template< ptrdiff_t ... StaticExtents >
  class mapping {
  public:

    // domain types

    using index_type = ptrdiff_t ;

    // constructors, copy, assignment, and destructor

    ~mapping() noexcept = default ;
    constexpr mapping() noexcept = default ;
    constexpr mapping(mapping const&) noexcept = default ;
    mapping& operator=(mapping const&) noexcept = default ;

    // observers of domain

    static constexpr int rank() noexcept;
    static constexpr int rank_dynamic() noexcept;

    static constexpr index_type static_extent(int) noexcept;

    constexpr index_type extent(int) const noexcept;

    constexpr index_type size() const noexcept;

    // observers of the codomain: [0..required_span_size())

    constexpr index_type required_span_size() const noexcept;

    // observers of the mapping from domain to codomain

    static constexpr bool is_always_unique     = /* deduced */ ;
    static constexpr bool is_always_contiguous = /* deduced */ ;
    static constexpr bool is_always_strided    = /* deduced */ ;

    constexpr bool is_unique() const noexcept;
    constexpr bool is_contiguous() const noexcept;
    constexpr bool is_strided() noexcept;

    constexpr index_type stride(int) const noexcept;
```

```
    // mapping domain index to access codomain element

    template< class ... IndexType >
    constexpr index_type operator()( IndexType ... indices ) const noexcept;
  };
};
```

`template< ptrdiff_t ... StaticExtents > class mapping`

Requires: Let `StaticExtents[ith]` be the ith member of the pack. `StaticExtents[ith] == std::dynamic_extent` or `0 <= StaticExtents[ith]`.

Effects: Defines the domain index space where `rank() == sizeof...(StaticExtents)` and each `StaticExtents[ith] == std::dynamic_extent` denotes that `ith` extent coordinate is a dynamic extent.

`constexpr mapping();`

Effects: If `static_extent(i) != std::dynamic_extent` then `extent(i) == static_extent(i)` otherwise `extent(i) == 0`.

`explicit constexpr mapping( index_type... ) noexcept;`
`explicit constexpr mapping(` *layout_property* `const&) noexcept;`

Constructors, assignment operators, and destructor requires and effects correspond to the corresponding members of `mdspan`.

```
static constexpr int rank() noexcept;
static constexpr int rank_dynamic() noexcept;
constexpr index_type size() const noexcept;
constexpr index_type extent(int) const noexcept;
constexpr index_type static_extent(int) noexcept;

template < class ... IndexType >
static constexpr index_type required_span_size( IndexType ... DynamicExtents ) noexcept;
static constexpr index_type required_span_size( *layout_property* const & ) noexcept;

static constexpr bool is_always_unique     = /* deduced */ ;
static constexpr bool is_always_contiguous = /* deduced */ ;
static constexpr bool is_always_strided    = /* deduced */ ;

constexpr bool is_unique() const noexcept;
constexpr bool is_contiguous() const noexcept;
constexpr bool is_strided() noexcept;

constexpr index_type stride(int) const noexcept;
```

Domain, codomain, and mapping observers requires and effects correspond to the corresponding members of `mdspan`.

`constexpr index_type required_span_size() const noexcept;`

Returns: The upper bound of the codomain one dimensional index space `[0..required_span_size())`.

```
template< class ... IndexType >
constexpr index_type operator()(IndexType ... indices) const noexcept;
```

Requires: `rank() == sizeof...(indices)` and `0 <= indices[ith] < extent(ith)`.

Returns: Result of mapping `indices...` from the domain multidimensional index space to the codomain one dimensional index space `[0..required_span_size())`.

4 Appendix: Preferred declaration syntax for multi-index space domain (P0332)

The proposed declaration mechanism for the multi-index domain space is verbose and unwieldy.

```
using tensor = std::mdspan<double,std::extents<std::dynamic_extent,std::dynamic_extent,std::dynamic_extent>> ;
```

The preferred mechanism is compact, is intuitive, LEWG has staw-polled strong preference, and users have voiced strong expressed preference.

```
using tensor = mdspan<double[][][]> ;
```

However, this syntax requires the non-functional language change proposed in P0332 to relax the definition of an incomplete array type.

Precedence:

There is precedence for using incomplete array types for dynamic extents.

- `std::shared_ptr<T[]>` and `std::unique_ptr<T[]>` denote a dynamic extent array through the incomplete type `T[]`
- P0674 denotes `make_shared<T[][N1][N2]>` to allocate a `shared_ptr` to a C style multidimensional array.

4.1 Impact on this proposal

`DataType`

> Requires: Is a complete or incomplete array type (8.3.4.p3). Each omitted static extent in the incomplete array type, `[]`, denotes a *dynamic* extent.
>
> `using element_type = std::remove_all_extents<DataType>::type ;`
>
> `constexpr int rank() const { return std::rank<DataType>::value ; }`
>
> `static_extent(i)` is `std::extent_v<DataType,i>`
>
> A dynamic extent is denoted when `std::extent_v<DataType,i> == 0`.
>
> The need for the magic number `std::dynamic_extent` is removed.

5 Appendix: Alignment or Merging with P0122 span (P0456)

A minor revision of P0122 `span` is proposed in P0456 that would more closely align `span` with `mdspan` and enable `span` to have a similar extensibility for access properties.

```
template< typename DataType , class ... Properties >
class span {
public:
  // change element_type declaration:
  using element_type = std::remove_extent_t< DataType > ;

};
```

Given P0456 the proposed `span` and `mdspan` could be merged into a single template class by simply requiring that all members specific to a one-dimensional span **Requre** that `rank() == 1`.

6 Appendix: Reference is potentially a proxy (P0860)

The `reference` type may be a proxy for accessing an `element_type` object. For example, if an `atomic_access` property were defined with the meaning that all access operations on codomain objects are atomic then the `reference` type must be an atomic reference type (paper P0019).

```
mdspan<int[],atomic_access> a( ptr , N );

static_assert( std::is_same_v< delctype(a(i)) , atomic_ref<int> > );
```

7 Appendix: Anticipated mdspan properties

```
namespace std {
namespace experimental {

  // bounds checking property
  template< bool Enable >
  struct bounds_check_if ;

  using bounds_check = bounds_check_if< true > ;
}}
```

> When `mdspan Properties...` includes `bounds_check_if<true>` then the mapping operators `mdspan::operator()` and

`mdspan::operator[]` verify that each index is valid, `0 <= index[ith] < extent(ith)`. Verification failure shall be reported.

8 Appendix: Examples

Given `mdspan x` then:

```
int d = 0 ;
index_type s = 1 ;
for ( int i = 0 ; i < x.rank() ; ++i ) {
  if ( x.static_extent(i) == std::dynamic_extent ) { ++d ; }
  s *= x.extent(i);
}
assert( d == x.rank_dynamic() );
assert( s == x.size() );
```

Subspan behavior:

```
// given U.rank() == 4
void foo( mdspan< DataType , Properties ... > const & U )
{
  auto V = subspan( U , make_pair(1,U.extent(0)-1) , 1 , make_pair(2,U.extent(2)-2 );
  assert( V.extent(0) == U.extent(0) - 2 );
  assert( V.extent(1) == U.extent(2) - 2 );
  assert( & V(0,0) == U(1,1,2,2) );
  assert( & V(1,0) == U(2,1,2,2) );
  assert( & V(0,1) == U(1,1,3,2) );
}
```

9 Related papers

ISOCPP issue: https://issues.isocpp.org/show_bug.cgi?id=80

- **P0122 : span: bounds-safe views for sequences of objects** The `mdspan` codomain concept of *span* is well-aligned with this paper.
- **P0367 : Accessors** The P0367 Accessors proposal includes polymorphic mechanisms for accessing the memory an object or span of objects. The `Properties...` extension point in this proposal is intended to include such memroy access properties.
- **P0454 : Wording for a Minimal ``mdspan``** (withdrawn)
- **P0546 : Preparing ``span`` for the future**
- **P0567 : Asynchronous Managed Pointer for Hetergeneous ...**
- **P0687 : Data Movement in C++**